



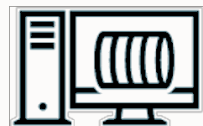
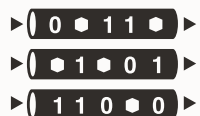
ORACLE

OCI Queue Service ご紹介

日本オラクル株式会社
2025年5月



メッセージ基盤の分類



分散ストリーミング

- 点と(複数の)点を接続
- ブローカー通信
- リアルタイム/高スループット
- メッセージの再利用性
- 順序保証
- 例) OCI Streaming (Kafka)

メッセージブローカー

- 点と(複数の)点を接続
- ブローカー通信
- Pushベース/非同期
- 緩い順序保証
- 例) Oracle TEQ (AQ)

メッセージキュー

- 点と点を接続
- ブローカー通信
- 信頼度の高い非同期通信
- 一度だけのメッセージ処理
- 処理の保証
- 複数のプロトコル
- 自動スケール
- 例) OCI Queue

(ストリーミング) Web サービス

- 点と点を接続
- 直接の通信
- 同期通信
- 例) HTTP Web Services, gRPC



OCI Queue Service

非同期メッセージングをサーバレスに実現する、ハイパフォーマンス・メッセージキュー

■ ユースケース

- アプリケーションを分離し、疎結合なアーキテクチャを構築
- 信頼性の高いスケーラブルなメッセージ基盤の活用

■ 特徴

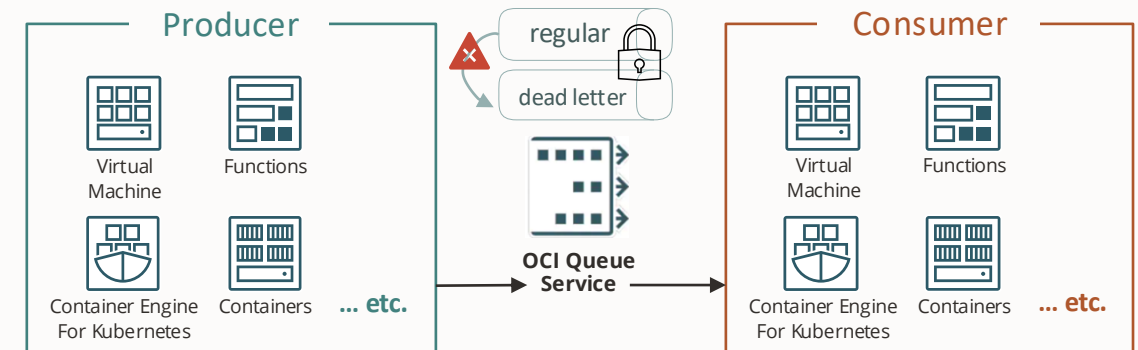
- フルマネージド、オートスケール
- OCI Streaming を補完する適用領域
 - オフセット(カーソル)無し、メッセージ消費は逐次確認
 - パーティション無し、需要に応じてオートスケール
- at-least-once 配信、配信順序はベストエフォート
- Dead Letter Queue (DLQ) の提供 (異常終了メッセージを退避)
- チャンネル機能(一時的なキュー内の宛先)
- キューメッセージの暗号化
- STOMPプロトコルをサポート

■ 価格

最初の100万リクエストは無料、移行100万リクエストごとに¥34.1

* 1リクエストのメッセージ容量が64KBを超える場合、64KBを1単位として複数のリクエストとして計算

** 複数メッセージの同時取得は、メッセージ容量に基づいて計算



- STOMP: Simple Text Orientated Messaging Protocol
軽量なメッセージングプロトコル

OCI Queue Service の特長



マネージド

自動化、メンテナンス要らず
のインフラとプラットフォーム



セキュア

外部/内部の脅威から
データを守る



スケール

停止なしの自動スケール
と最低限のコスト



従量課金

最大限のコストパフォーマンス



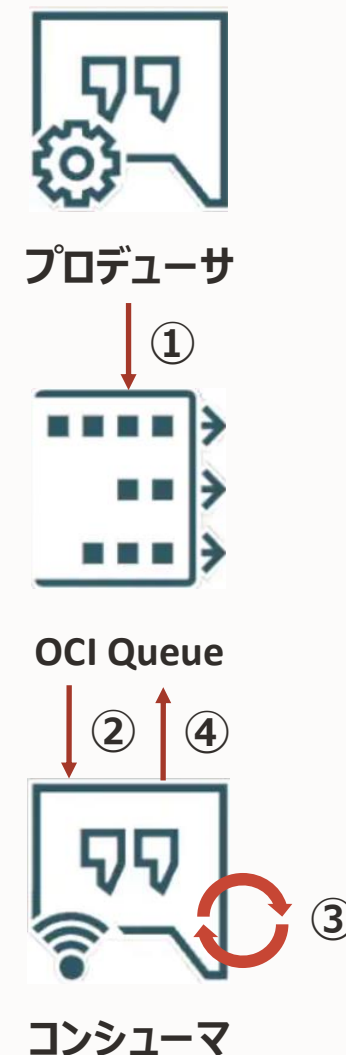
複数のプロトコル

HTTP/STOMPのサポート



OCI Queue Service の基本的な処理の流れ

- ①プロデューサ(メッセージ送信クライアント)はメッセージを OCI Queue に公開
- ②コンシューマ(メッセージ受信クライアント)が OCI Queue 上のメッセージを取得
- ③コンシューマが取得したメッセージを処理
- ④コンシューマがOCI Queueに対して処理したメッセージの削除をリクエスト



OCI Queue Service の機能

at-least-once デリバリーを採用

at-most-once デリバリー

- メッセージ毎に、そのメッセージは0回か1回配信される
- メッセージは重複しない
- メッセージが失われる可能性がある

at-least-once デリバリー

- メッセージ毎に、少なくとも1回成功するように潜在的に複数回配信が行われる
- メッセージの重複があり得る
- メッセージの複製が失われない

exactly-once デリバリー

- メッセージ毎に正確に1回の配信が受信者に対して行われる
- メッセージは重複しない
- メッセージは失われたり複製されたりしない

OCI Queue Service の機能

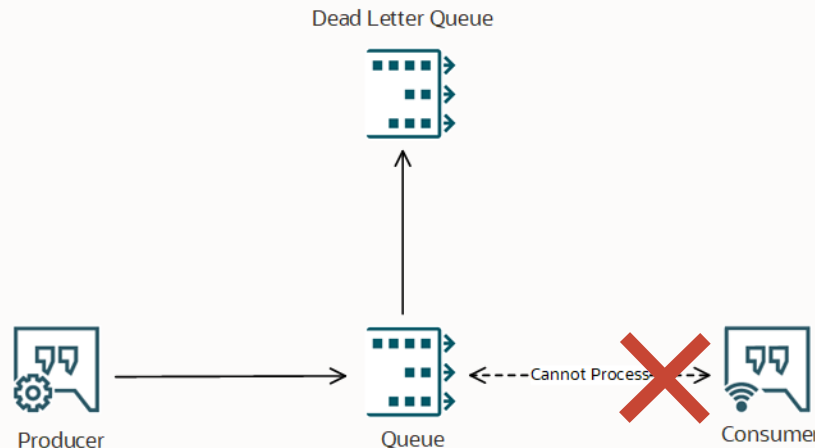
デッド・レター・キュー (DLQ)

正常に処理されていないメッセージを格納する方法として、デッド・レター・キューを提供

- メッセージがキューから取得されるたびに、メッセージの配信回数が増加
- メッセージの配信回数が設定値を超えると、メッセージはデッド・レター・キューに移動
 - キューの設定値(最大配信試行回数)はキューの作成時または変更時に指定可能

デッド・レター・キューに転送されたメッセージが削除されるのは以下の2パターン

- 設定した保存期間が過ぎ、サービスによって削除
- メッセージを処理後のコンシューマなどによって手動で削除

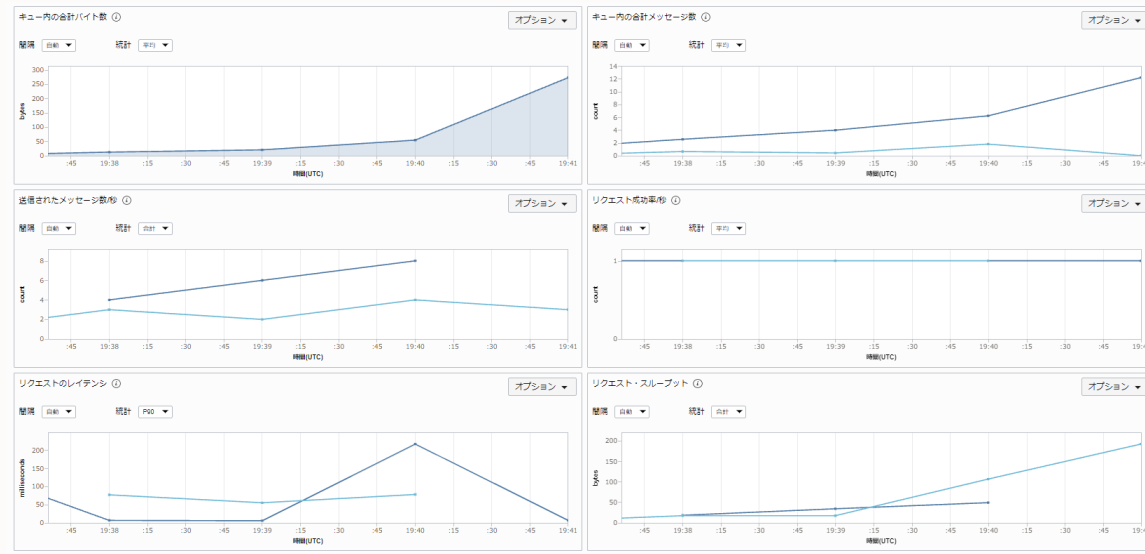


OCI Queue Service の機能

Queueのモニタリング

キューとデッドレター・キューの様々なメトリックを取得可能

- 消費可能なキュー内の現在のメッセージの概数
- コンシューマに配信されたがまだ削除されていないメッセージの概数
- 表示メッセージおよび処理中メッセージのサイズの合計としてのキューの概算サイズ(バイト)
- etc.

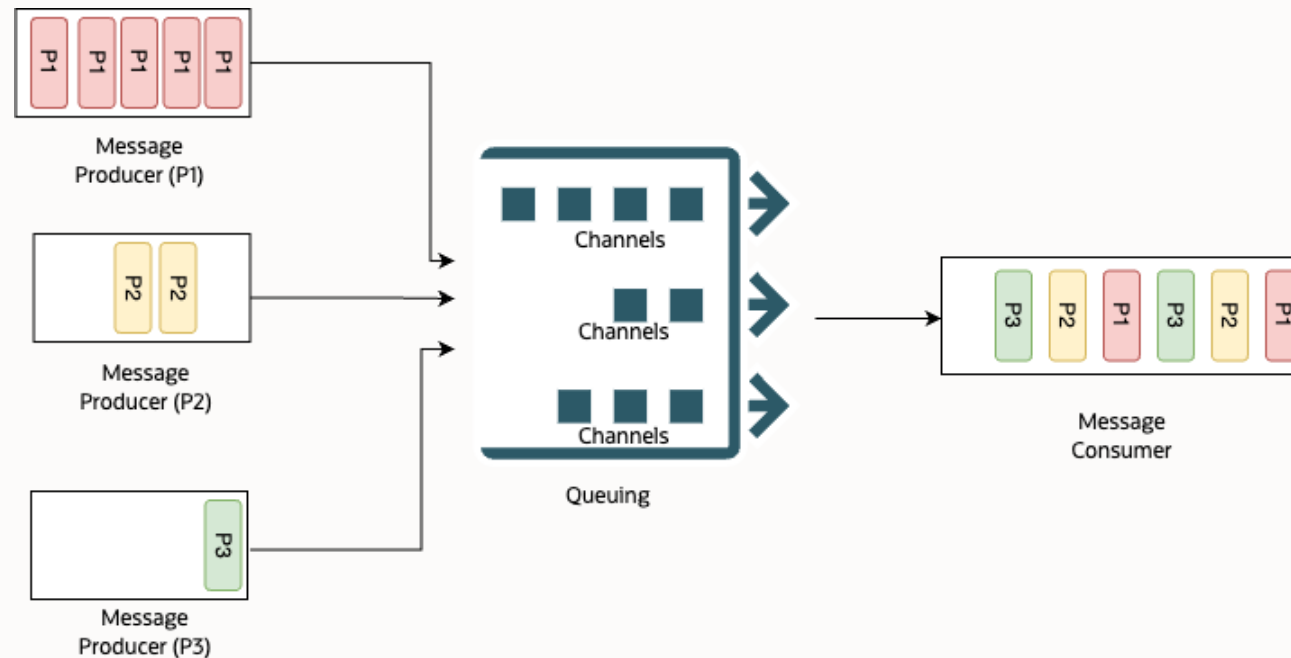


OCI Queue Service の機能

チャンネル

一時的なキュー内の宛先として機能する

- プロデューサはメッセージを特定のチャンネルに公開可能
- コンシューマはメッセージを取得するチャンネルのフィルタリングが可能
 - 複数のチャンネルがフィルタリングされたときには、その複数のチャンネルからランダムに選択されメッセージを取得(ランダム・チャンネル)
 - チャンネルのフィルタリングをしない場合はキュー・レベルでランダム・チャンネルから取得

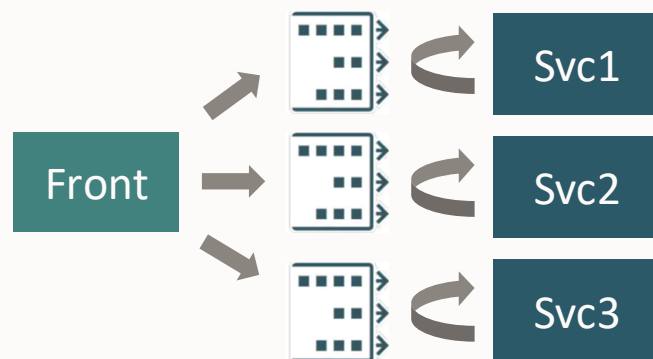


OCI Queue Service のユースケース

アプリケーションの分離

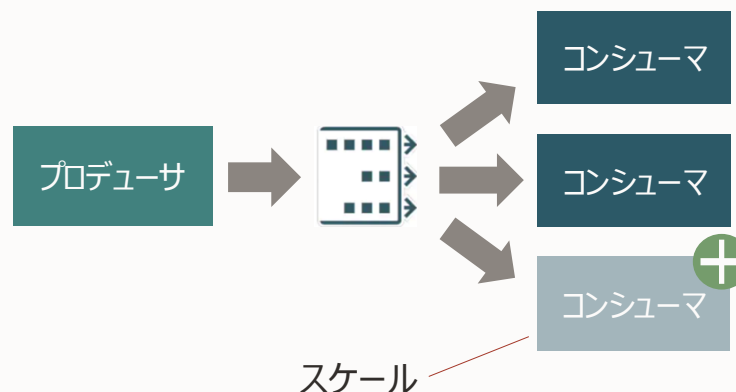
フロントエンドとバックエンドの分離

- バックエンドサービスの並列化
 - 非同期処理の実現
 - システム利用ユーザはバックエンドの処理終了を待たずに結果を得る
- 疎結合性による開発スピードの向上



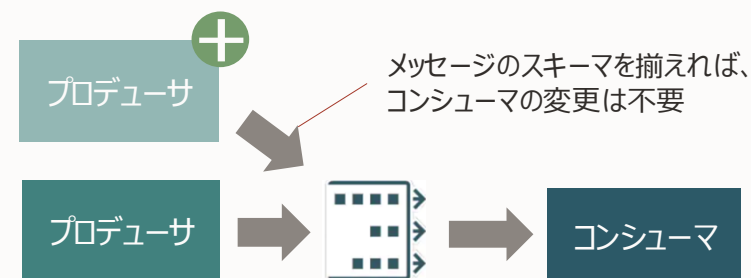
コンポーネントの独立したスケール

- ボトルネックの解消
 - システム内でボトルネックの部分 (IaaS/コンテナ/DB/etc.)のみを必要なだけスケール
- 余剰リソースの削減



システムの拡張性

- 後からコンポーネントの追加(機能の追加など)が容易
- QueueにProduce/QueueからConsumeという形を守ればよい

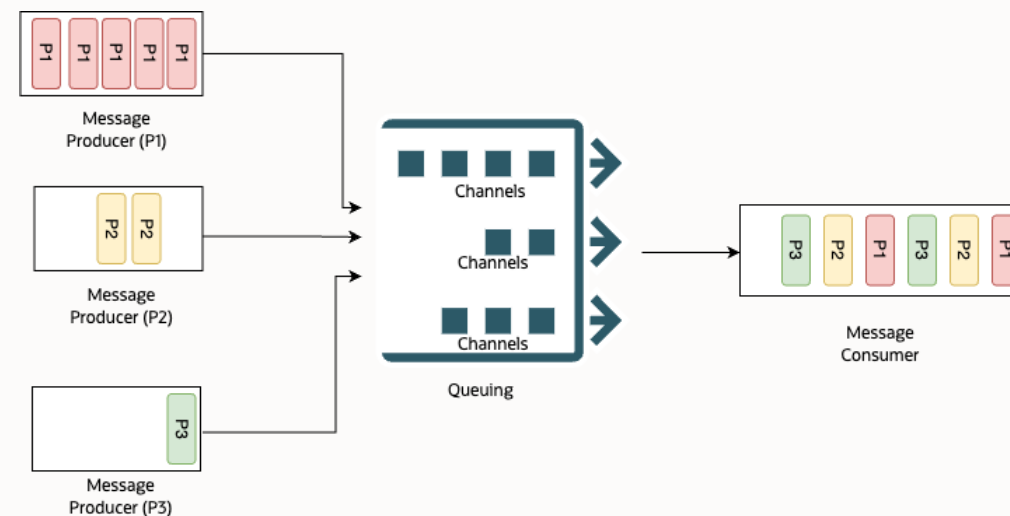
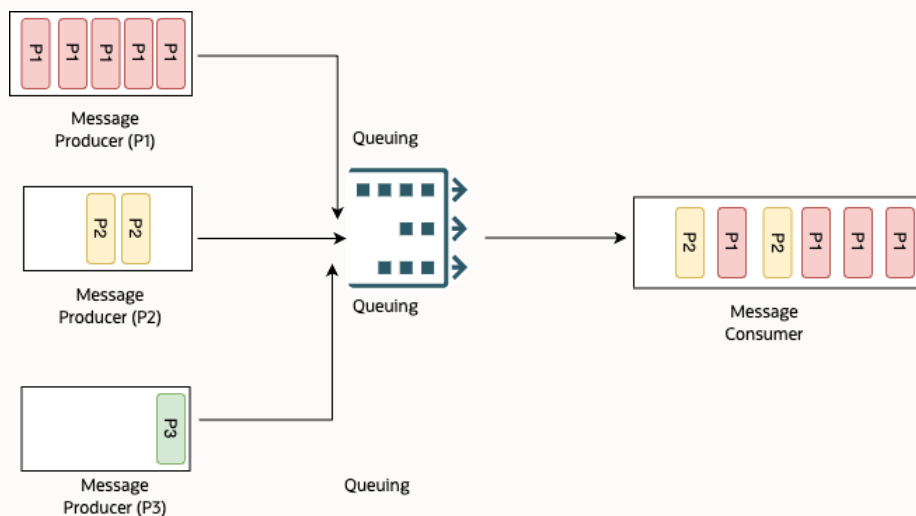


チャネルを用いたOCI Queueのユースケース

ユースケース: メッセージ処理の公平性

複数のプロデューサが同じキューにメッセージをPublishする場合に、あるプロデューサ(P1)からのメッセージが突然急増すると、他のプロデューサからのメッセージのConsumeが大幅に遅延する

チャネル機能を利用すると、キュー・レベルのランダム・チャネルからメッセージを取得できるため、メッセージ処理の偏りをなくし、公平性を高めることができる

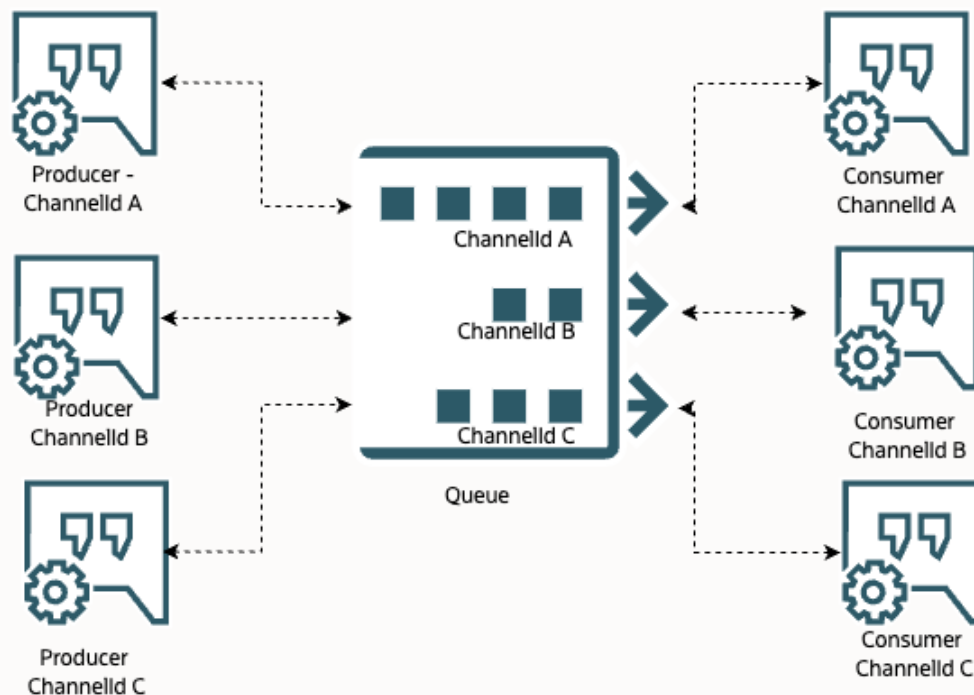


チャンネルを用いたOCI Queueのユースケース

ユースケース: 一時キューとして利用(リクエスト/レスポンス・パターン)

本来Pub/Subモデルの非同期メッセージングでは、リクエストとレスポンスは切り離されたものとなる

- レスポンス先をリクエスト元に正確に返却するには、リクエスト元の特定制が必要
- 一時キューとしてチャンネルを活用することで、特定制が可能



OCI Queue Service の制限

デフォルトの制限

リソース	詳細
キュー	テナンシ当たり10/リージョン
最大PutMessageリクエスト・サイズ	512KBおよび20メッセージ
最大GetMessageレスポンス・サイズ	2MBおよび20メッセージ
メッセージの最大容量	128KB
処理中メッセージの最大数	キュー当たり100,000
キューごとの最大メッセージ	無制限
メッセージの保存	最大：7日間 最小：10秒 デフォルト：1日
メッセージ表示タイムアウト	最大：12時間 最小：メッセージ・レベルで0秒、キュー・レベルで1秒 デフォルト：30秒



OCI Queue Service の制限(続き)

デフォルトの制限

リソース	詳細
最大同時GETリクエスト	キュー当たり1,000リクエスト/秒
最大メッセージ操作数	キュー当たりのAPI当たり1,000リクエスト/秒
最大データレート	キュー当たりのイングレス: 10MB/s キュー当たりのエグレス: 10MB/s
投票タイムアウト	最大: 30秒 最小: 0秒
STOMPスループット	STOMP接続あたり10Mバイト/秒
ストレージ	テナンシ当たり20GB、キュー当たり2GB



顧客事例：株式会社アプルーシッド様

「ドクセル」を、OCI Container Instancesを活用してGoogle Cloudから移行し、圧倒的なコスト削減を実現

ドクセル（Docswell）

- PDFやパワーポイントのスライドを共有できる国産のスライドシェアサービス。ユーザー約30万人、月間アクセス約600万回。

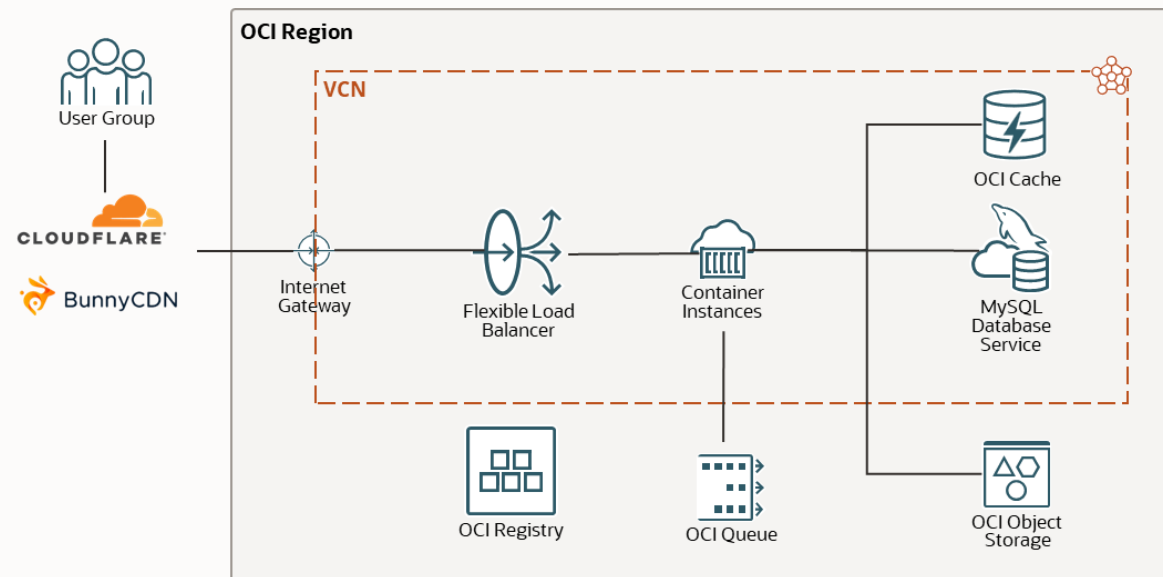
従来の課題

- 既存のGoogle Cloud環境のコスト削減の必要性があった(特に、アウトバウンドのデータ転送のコスト)。
- ユーザーアクセス時にコンテナが都度起動するスピンアップの仕組みにより、時に遅延が発生していた。

採用ポイントと導入効果

- 既存Google Cloud環境から移行し、特にデータ転送が毎月10TB無料の利点を享受し、全体で**約65%のコスト削減**を実現。MySQL Database Serviceの高可用性構成の部分は、要件を実現しつつ約50%のコスト削減を実現。
- OCI Container Instancesを活用し、**Kubernetesを利用せずに既存のコンテナアプリケーションをシームレスに移行**できた。
- コスト面で有利なContainer Instancesを必要個数、常時起動しておくことで、スピンアップの仕組みにする必要がなくなった。結果、**遅延のない安定的な処理**を実現。
- 複数のコンテナ間の**疎結合な処理にOCI Queueを活用**。

システム構成イメージ



利用サービス（クラウドサービス／その他）

- OCI Container Instances
- OCI Registry
- MySQL Database Service
- OCI Queue
- OCI Cache
- コスト削減フレームワーク (Oracleによる移行アセスメント支援)



ORACLE